

University of Pittsburgh

Introduction to Operating Systems
CS/COE 1550

Spring 2022

Sherif Khattab

ksm73@pitt.edu

(Some slides are from **Silberschatz, Galvin and Gagne ©2013)**

Announcements

- Upcoming deadlines:
  - Project 1: due on 2/18
  - Homework 5: due 2/21
  - Lab 2: due on 2/28

Previous lecture …

- Dining philosophers
- Deadlock prevention

Muddiest Points

- Checked on Tophat

Questions of the Day

How to detect deadlocks?

How to avoid deadlocks?

Banker's Algorithm

We can use the same algorithm for both detecting and avoiding deadlocks

Deadlock detection algorithm

	A	B	C	D
Avail	2	3	0	1

Hold

Process	A	B	C	D
1	0	3	0	0
2	1	0	1	1
3	0	2	1	0
4	2	2	3	0

Want

Process	A	B	C	D
1	3	2	1	0
2	2	2	0	0
3	3	5	3	1
4	0	4	1	1

```
current=avail;
for (j = 0; j < N; j++) {
 for (k=0; k<N; k++) {
  if (finished[k])
   continue;
  if (want[k] <= current) {
   finished[k] = 1;
   current += hold[k];
   break;
  }
 }
 if (k==N) {
   printf "Deadlock!\n";
   // finished[k]==0 means process is in
   // the deadlock
   break;
 }
}
```

Note: want[j], hold[j], current, avail are arrays!

Deadlock detection algorithm

	A	B	C	D
current	2	3	0	1

Hold

Process	A	B	C	D
1	0	3	0	0
2	1	0	1	1
3	0	2	1	0
4	2	2	3	0

Want

Process	A	B	C	D
1	3	2	1	0
2	2	2	0	0
3	3	5	3	1
4	0	4	1	1

```
current=avail;
for (j = 0; j < N; j++) {
 for (k=0; k<N; k++) {
  if (finished[k])
   continue;
  if (want[k] <= current) {
   finished[k] = 1;
   current += hold[k];
   break;
  }
 }
 if (k==N) {
   printf "Deadlock!\n";
   // finished[k]==0 means process is in
   // the deadlock
   break;
 }
}
```

Note: want[j], hold[j], current, avail are arrays!

P_2

Deadlock detection algorithm

	A	B	C	D
current	2	3	0	1

Hold

Process	A	B	C	D
1	0	3	0	0
2	1	0	1	1
3	0	2	1	0
4	2	2	3	0

Want

Process	A	B	C	D
1	3	2	1	0
2	2	2	0	0
3	3	5	3	1
4	0	4	1	1

```
current=avail;
for (j = 0; j < N; j++) {
 for (k=0; k<N; k++) {
  if (finished[k])
   continue;
  if (want[k] <= current) {
   finished[k] = 1;
   current += hold[k];
   break;
  }
 }
 if (k==N) {
   printf "Deadlock!\n";
   // finished[k]==0 means process is in
   // the deadlock
   break;
 }
}
```

Note: want[j], hold[j], current, avail are arrays!

P_2

Deadlock detection algorithm

	A	B	C	D
current	3	3	1	2

Hold

Process	A	B	C	D
1	0	3	0	0
2	1	0	1	1
3	0	2	1	0
4	2	2	3	0

Want

Process	A	B	C	D
1	3	2	1	0
✓ 2	2	2	0	0
3	3	5	3	1
4	0	4	1	1

```
current=avail;
for (j = 0; j < N; j++) {
 for (k=0; k<N; k++) {
  if (finished[k])
   continue;
  if (want[k] <= current) {
   finished[k] = 1;
   current += hold[k];
   break;
  }
 }
 if (k==N) {
   printf "Deadlock!\n";
   // finished[k]==0 means process is in
   // the deadlock
   break;
 }
}
```

Note: want[j], hold[j], current, avail are arrays!

P_2

Deadlock detection algorithm

	A	B	C	D
current	3	3	1	2

Hold

Process	A	B	C	D
1	0	3	0	0
2	1	0	1	1
3	0	2	1	0
4	2	2	3	0

Want

Process	A	B	C	D
1	3	2	1	0
✓ 2	2	2	0	0
3	3	5	3	1
4	0	4	1	1

```
current=avail;
for (j = 0; j < N; j++) {
 for (k=0; k<N; k++) {
  if (finished[k])
   continue;
  if (want[k] <= current) {
   finished[k] = 1;
   current += hold[k];
   break;
  }
 }
 if (k==N) {
   printf "Deadlock!\n";
   // finished[k]==0 means process is in
   // the deadlock
   break;
 }
}
```

Note: want[j], hold[j], current, avail are arrays!

P_2

Deadlock detection algorithm

	A	B	C	D
current	3	3	1	2

Hold

Process	A	B	C	D
1	0	3	0	0
2	1	0	1	1
3	0	2	1	0
4	2	2	3	0

Want

Process	A	B	C	D
1	3	2	1	0
✓ 2	2	2	0	0
3	3	5	3	1
4	0	4	1	1

```
current=avail;
for (j = 0; j < N; j++) {
 for (k=0; k<N; k++) {
  if (finished[k])
   continue;
  if (want[k] <= current) {
   finished[k] = 1;
   current += hold[k];
   break;
  }
 }
 if (k==N) {
   printf "Deadlock!\n";
   // finished[k]==0 means process is in
   // the deadlock
   break;
 }
}
```

Note: want[j], hold[j], current, avail are arrays!

P_2

Deadlock detection algorithm

	A	B	C	D
current	3	3	1	2

Hold

Process	A	B	C	D
1	0	3	0	0
2	1	0	1	1
3	0	2	1	0
4	2	2	3	0

Want

Process	A	B	C	D
1	3	2	1	0
2	2	2	0	0
3	3	5	3	1
4	0	4	1	1

```
current=avail;
for (j = 0; j < N; j++) {
 for (k=0; k<N; k++) {
  if (finished[k])
   continue;
  if (want[k] <= current) {
   finished[k] = 1;
   current += hold[k];
   break;
  }
 }
 if (k==N) {
   printf "Deadlock!\n";
   // finished[k]==0 means process is in
   // the deadlock
   break;
 }
}
```

Note: want[j], hold[j], current, avail are arrays!

P_2 P_1

Deadlock detection algorithm

	A	B	C	D
current	3	6	1	2

Hold

Process	A	B	C	D
1	0	3	0	0
2	1	0	1	1
3	0	2	1	0
4	2	2	3	0

Want

Process	A	B	C	D
1	3	2	1	0
2	2	2	0	0
3	3	5	3	1
4	0	4	1	1

```
current=avail;
for (j = 0; j < N; j++) {
 for (k=0; k<N; k++) {
  if (finished[k])
   continue;
  if (want[k] <= current) {
   finished[k] = 1;
   current += hold[k];
   break;
  }
 }
 if (k==N) {
   printf "Deadlock!\n";
   // finished[k]==0 means process is in
   // the deadlock
   break;
 }
}
```

Note: want[j], hold[j], current, avail are arrays!

P_2 P_1

Deadlock detection algorithm

	A	B	C	D
current	3	6	1	2

Hold

Process	A	B	C	D
1	0	3	0	0
2	1	0	1	1
3	0	2	1	0
4	2	2	3	0

Want

Process	A	B	C	D
1	3	2	1	0
2	2	2	0	0
3	3	5	3	1
4	0	4	1	1

```
current=avail;
for (j = 0; j < N; j++) {
 for (k=0; k<N; k++) {
  if (finished[k])
   continue;
  if (want[k] <= current) {
   finished[k] = 1;
   current += hold[k];
   break;
  }
 }
 if (k==N) {
   printf "Deadlock!\n";
   // finished[k]==0 means process is in
   // the deadlock
   break;
 }
}
```

Note: want[j], hold[j], current, avail are arrays!

P_2 P_1 P_4

Deadlock detection algorithm

	A	B	C	D
current	5	8	4	2

Hold

Process	A	B	C	D
1	0	3	0	0
2	1	0	1	1
3	0	2	1	0
4	2	2	3	0

Want

Process	A	B	C	D
1	3	2	1	0
2	2	2	0	0
3	3	5	3	1
4	0	4	1	1

```
current=avail;
for (j = 0; j < N; j++) {
 for (k=0; k<N; k++) {
  if (finished[k])
   continue;
  if (want[k] <= current) {
   finished[k] = 1;
   current += hold[k];
   break;
  }
 }
 if (k==N) {
   printf "Deadlock!\n";
   // finished[k]==0 means process is in
   // the deadlock
   break;
 }
}
```

Note: want[j], hold[j], current, avail are arrays!

P_2 P_1 P_4

Deadlock detection algorithm

	A	B	C	D
current	5	8	4	2

Hold

Process	A	B	C	D
1	0	3	0	0
2	1	0	1	1
3	0	2	1	0
4	2	2	3	0

Want

Process	A	B	C	D
1	3	2	1	0
2	2	2	0	0
3	3	5	3	1
4	0	4	1	1

```
current=avail;
for (j = 0; j < N; j++) {
 for (k=0; k<N; k++) {
  if (finished[k])
   continue;
  if (want[k] <= current) {
   finished[k] = 1;
   current += hold[k];
   break;
  }
 }
 if (k==N) {
   printf "Deadlock!\n";
   // finished[k]==0 means process is in
   // the deadlock
   break;
 }
}
```

Note: want[j], hold[j], current, avail are arrays!

P_2 P_1 P_4 P_3

Deadlock detection algorithm

	A	B	C	D
current	5	10	5	2

Hold

Process	A	B	C	D
1	0	3	0	0
2	1	0	1	1
3	0	2	1	0
4	2	2	3	0

Want

Process	A	B	C	D
1	3	2	1	0
2	2	2	0	0
3	3	5	3	1
4	0	4	1	1

```
current=avail;
for (j = 0; j < N; j++) {
 for (k=0; k<N; k++) {
  if (finished[k])
   continue;
  if (want[k] <= current) {
   finished[k] = 1;
   current += hold[k];
   break;
  }
 }
 if (k==N) {
   printf "Deadlock!\n";
   // finished[k]==0 means process is in
   // the deadlock
   break;
 }
}
```

$\Theta(N^2M)$

Note: want[j], hold[j], current, avail are arrays!

P_2 P_1 P_4 P_3

Deadlock detection algorithm

	A	B	C	D
current	5	10	5	2

Hold

Process	A	B	C	D
1	0	3	0	0
2	1	0	1	1
3	0	2	1	0
4	2	2	3	0

Want

Process	A	B	C	D
✓ 1	3	2	1	0
✓ 2	2	2	0	0
✓ 3	3	5	3	1
✓ 4	0	4	1	1

```
current=avail;
for (j = 0; j < N; j++) {
 for (k=0; k<N; k++) {
  if (finished[k])
   continue;
  if (want[k] <= current) {
   finished[k] = 1;
   current += hold[k];
   break;
  }
 }
 if (k==N) {
   printf "Deadlock!\n";
   // finished[k]==0 means process is in
   // the deadlock
   break;
 }
}
```

No deadlock

Note: want[j], hold[j], current, avail are arrays!

P_2 P_1 P_4 P_3

Banker's Algorithm Insights

- It is possible that some event sequences lead a deadlock
- What we are looking for is at least one event sequence that can make all processes finish
- If such sequence exists, the state is safe
- The Banker's algorithm finds such sequence if it exists

Using the Banker's Algorithm for Deadlock Avoidance

- Call the algorithm on the following ``What-if" state instead of the current state

avoiding deadlocks

Request from Process i

avail` = avail - Request

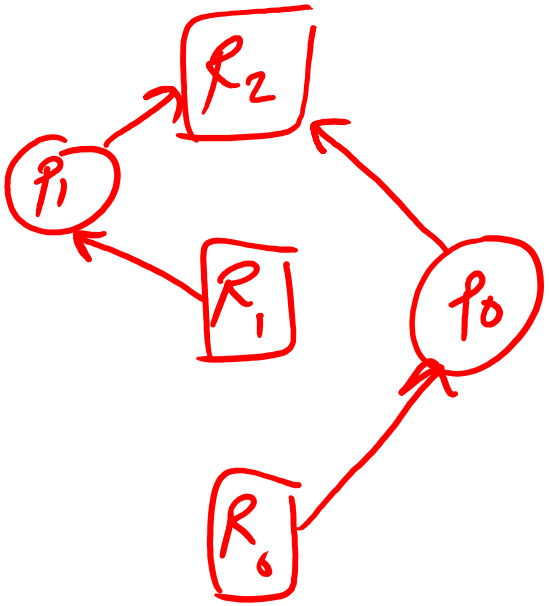
hold`[i] = hold[i] + Request

Want`[i] = want[i] - Request

Run algo on avail`, hold`, Want`

Proof sketch for Deadlock Prevention

- If resources are ordered and resource requests within each process follow the resource ordering, the resource allocation graph will have no downward arrows.



no ↓ ⇒ no cycles

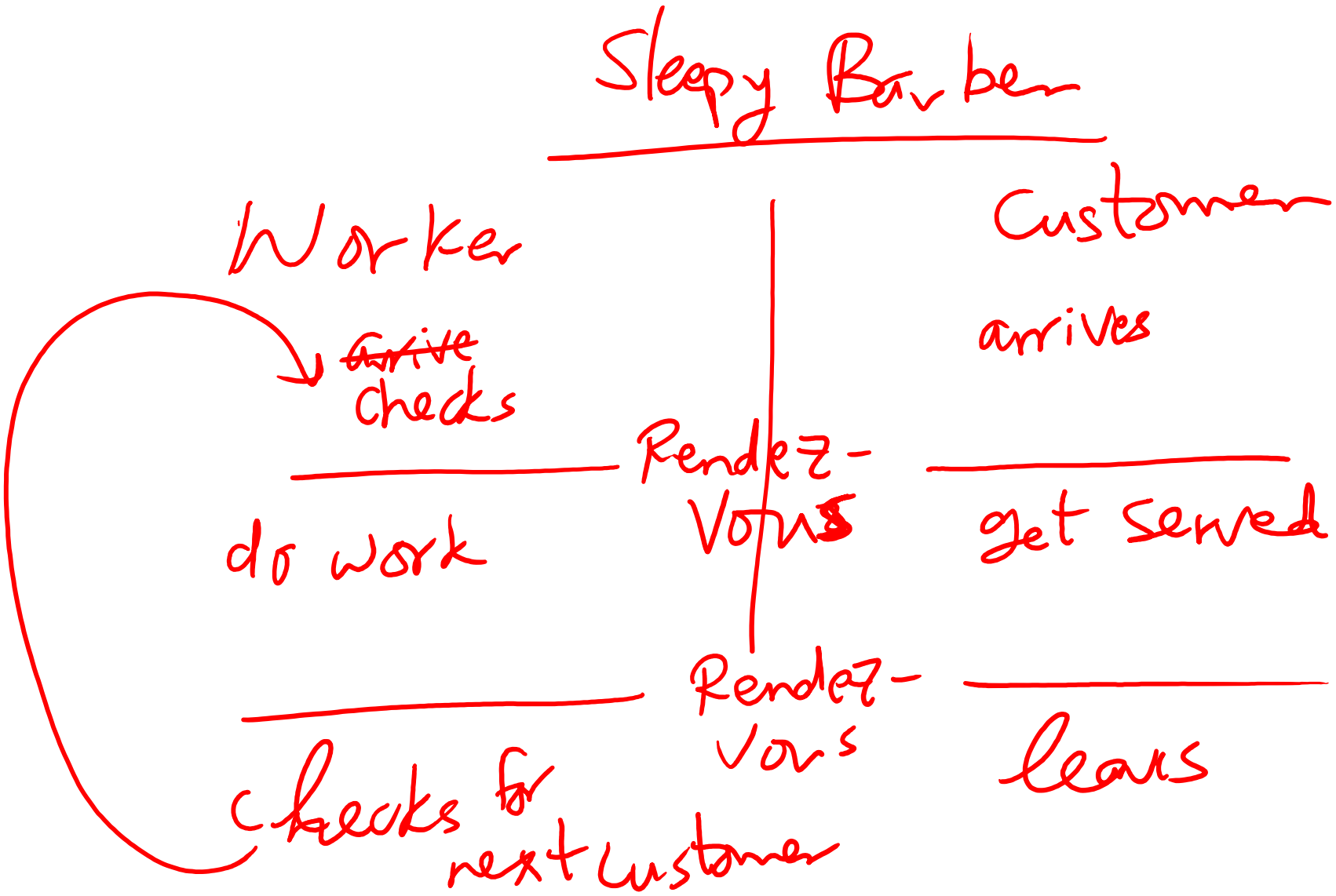
P0 holds R0 Waits R2

P1 holds R1 Waits R2

Problem of the Day: Sleepy Barbers

- We have two sets of processes
  - Worker processes (e.g., barbers)
  - Customer processes
- Customer processes may arrive at anytime
- Worker processes check in when they are not serving any customers
- Each worker process must wait until it gets matched with a customer process
- Each customer process must wait until it gets matched with a worker process
- The customer process cannot leave until the matched worker process finishes the work
- The worker process cannot check in for the next customer until the matched customer process leaves
- Many applications in the real-world

Rendezvous Pattern



Solution Using Semaphores: Take 1

- One pair of semaphores per rendezvous
  - RV1a and RV1b
  - RV2a and RV2b
- Notice the flipped order of the down and up calls in the two processes

Worker

Semaphores RV1a, RV1b (0) (0)

RV2a, RV2b (0) (0)

arrives/checks in

down(RV1a)

up(RV1b)

does work

up(RV2a)

down(RV2b)

Customer

arrives

up(RV1a)

down(RV1b)

gets served

down(RV2a)

up(RV2b)

Solution Using Semaphores: Take 1

- This solution doesn't work for multiple workers and multiple customers
- A customer can leave before its associated worker finishes